CAPÍTULO 7. PERSONALIZACIÓN DE GIT

Introducción

Git permite adaptar su funcionamiento a las necesidades y preferencias de cada usuario. Para ello, ofrece diferentes opciones de configuración que permiten establecer datos personales, modificar el comportamiento de los comandos, definir alias y personalizar aspectos relacionados con la edición de archivos y la visualización de la información.

La personalización facilita el trabajo diario, reduce la repetición de comandos y contribuye a mantener una forma de trabajo consistente entre diferentes proyectos. En este capítulo se explican los principales mecanismos de configuración de Git, los niveles en los que se pueden aplicar, la creación de alias y algunas opciones útiles para mejorar la experiencia de uso.

1. Configurar la identidad del usuario

Cuando se registra una confirmación o *commit*, Git almacena información sobre su autor. Por esta razón, es importante configurar el nombre y el correo electrónico que se utilizarán para identificar las confirmaciones.

Para establecer el nombre del usuario, se utiliza:

git config --global user.name "Nombre del usuario"

Para establecer el correo electrónico:

git config --global user.email "correo@ejemplo.com"

La opción --global aplica la configuración al usuario del sistema operativo, por lo que normalmente se utiliza para que estos datos estén disponibles en todos sus repositorios locales.

Si se necesita establecer una identidad diferente para un proyecto específico, se puede ingresar a la carpeta del repositorio y ejecutar los comandos sin la opción --global:

git config user.name "Nombre del proyecto"

git config user.email "proyecto@ejemplo.com"

La configuración local tiene prioridad sobre la configuración global para ese repositorio.

Es importante utilizar los datos de identidad adecuados, especialmente cuando se trabaja en proyectos académicos o colaborativos.

2. Consultar la configuración de Git

Git permite revisar los valores de configuración que están activos en el sistema.

Para consultar la lista de configuraciones disponibles, se utiliza:

git config --list

Este comando muestra las opciones de configuración que Git encuentra en los diferentes niveles. Algunos valores pueden aparecer más de una vez si existen configuraciones distintas.

Para consultar específicamente el nombre del usuario:

git config user.name

Para consultar el correo electrónico:

git config user.email

También es posible abrir el archivo de configuración global mediante:

git config --global --edit

Este comando abre el archivo de configuración en el editor asociado a Git. Desde allí se pueden revisar y modificar opciones, siempre que se tenga cuidado de mantener una sintaxis válida.

3. Niveles de configuración

Git dispone de diferentes niveles de configuración que determinan el alcance de cada ajuste.

- **Configuración local**

La configuración local se aplica únicamente al repositorio actual. Se almacena normalmente en el archivo .git/config, ubicado dentro de la carpeta del repositorio.

Se utiliza cuando un proyecto necesita opciones específicas que no deben afectar a los demás repositorios del usuario.

Ejemplo:

git config user.name "Usuario del proyecto"

- **Configuración global**

La configuración global se aplica a los repositorios del usuario del sistema operativo, salvo que una configuración de mayor prioridad la reemplace.

Ejemplo:

git config --global core.editor "notepad"

Este comando establece el Bloc de notas de Windows como editor de Git, siempre que esté disponible y pueda ejecutarse desde el entorno correspondiente.

- **Configuración del sistema**

La configuración del sistema puede aplicarse a todos los usuarios del computador. Generalmente, requiere permisos administrativos y se utiliza cuando se necesita establecer un comportamiento común para diferentes cuentas del equipo.

Ejemplo:

git config --system --list

Este comando permite consultar las opciones configuradas en ese nivel, siempre que el usuario tenga los permisos necesarios.

En términos generales, cuando una misma opción está definida en varios niveles, la configuración más específica suele prevalecer. La prioridad habitual es: configuración local, global y del sistema, aunque también existen otros mecanismos de configuración con reglas particulares.

4. **Crear alias para simplificar comandos**

Un alias permite asignar un nombre corto a un comando o conjunto de opciones. Esta función facilita el uso de Git y evita escribir instrucciones largas repetidamente.

Por ejemplo, para crear un alias llamado st que ejecute git status, se utiliza:

git config --global alias.st status

Después, se puede consultar el estado del repositorio con:

git st

También es posible crear un alias para consultar el historial de manera resumida:

git config --global alias.hist "log --oneline --graph --all"

Para utilizarlo:

git hist

Otro ejemplo consiste en crear un alias para mostrar las ramas existentes:

git config --global alias.br branch

Y ejecutarlo mediante:

git br

Los alias permiten adaptar Git a las preferencias del usuario. Sin embargo, se recomienda utilizar nombres fáciles de recordar y documentar los alias cuando se trabaja en equipo, para evitar confusiones entre colaboradores.

5. **Configurar el editor de texto**

Git puede necesitar un editor de texto para redactar mensajes de confirmación, editar mensajes de fusión o modificar determinados archivos de configuración.

Para establecer el Bloc de notas de Windows como editor, se puede utilizar:

git config --global core.editor "notepad"

La disponibilidad de esta opción depende del sistema operativo y de que el editor pueda ejecutarse desde la terminal.

En otros entornos, se pueden configurar editores como Visual Studio Code o Vim, según las preferencias del usuario.

Por ejemplo, para utilizar Visual Studio Code como editor de Git, si está instalado y su comando code está disponible en la terminal, se puede ejecutar:

git config --global core.editor "code --wait"

La opción --wait permite que Git espere a que se cierre el archivo en el editor antes de continuar con la operación correspondiente.

6. **Configurar el comportamiento de los finales de línea**

Los archivos de texto pueden utilizar diferentes convenciones para representar los finales de línea. En Windows suele encontrarse la combinación CRLF, mientras que en sistemas similares a Unix se utiliza LF.

Estas diferencias pueden generar modificaciones aparentes cuando un archivo pasa de un sistema operativo a otro.

Git dispone de la opción core.autocrlf, que permite controlar determinadas conversiones de finales de línea.

En Windows, una configuración habitual es:

git config --global core.autocrlf true

Esta opción permite convertir los finales de línea durante determinadas operaciones de entrada y salida.

Sin embargo, la configuración adecuada depende del sistema operativo y de las convenciones del proyecto. En proyectos colaborativos, también se puede utilizar un archivo .gitattributes para establecer reglas explícitas sobre el tratamiento de los archivos.

Por esta razón, no se recomienda cambiar esta opción sin comprender cómo puede afectar a los archivos del repositorio.

7. **Configurar colores en la terminal**

Git puede mostrar información con colores para facilitar la identificación de estados, diferencias y mensajes.

Para habilitar el uso de colores en la salida de Git, se puede ejecutar:

git config --global color.ui auto

Con esta configuración, Git utiliza colores cuando el entorno lo permite.

Los colores pueden ayudar a distinguir las líneas agregadas y eliminadas en las diferencias entre archivos, así como a interpretar con mayor facilidad determinados resultados de los comandos.

Esta opción es útil para quienes trabajan frecuentemente desde la terminal.

8. **Consultar la ayuda de Git**

Git incluye documentación de ayuda que permite conocer el funcionamiento de los comandos y sus opciones.

Para consultar la ayuda general:

git help

Para obtener información sobre un comando específico, por ejemplo config, se utiliza:

git help config

También se puede consultar una descripción breve mediante:

git config -h

La disponibilidad y la forma de mostrar la documentación pueden variar según la instalación. Si la ayuda no se abre correctamente, se puede consultar la documentación oficial en https://git-scm.com/docs.

La consulta de ayuda es una práctica recomendable porque permite verificar las opciones disponibles y comprender el alcance de los comandos antes de ejecutarlos.

Ejercicio práctico

El siguiente ejercicio permite aplicar las principales opciones de personalización de Git.

Paso 1. Consultar el nombre de usuario configurado:

git config user.name

Paso 2. Consultar el correo electrónico:

git config user.email

Paso 3. Si es necesario, establecer el nombre del usuario:

git config --global user.name "Nombre del estudiante"

Paso 4. Establecer el correo electrónico correspondiente:

git config --global user.email "correo@ejemplo.com"

Se deben reemplazar los datos de ejemplo por la información que se desea utilizar.

Paso 5. Crear un alias para consultar el estado del repositorio:

git config --global alias.st status

Paso 6. Ejecutar el alias:

git st

Paso 7. Crear un alias para consultar el historial:

git config --global alias.hist "log --oneline --graph --all"

Paso 8. Consultar el historial utilizando el alias:

git hist

Paso 9. Revisar las opciones de configuración:

git config --list

Al finalizar, el usuario habrá configurado su identidad, creado alias y consultado las opciones disponibles en Git.

Conclusión

La personalización de Git permite adaptar el sistema de control de versiones a las necesidades de cada usuario y proyecto. La configuración de la identidad, los niveles de configuración, los alias, el editor de texto y las opciones de visualización contribuyen a mejorar la organización y la eficiencia del trabajo.

Además, conocer los comandos de consulta de configuración y ayuda permite resolver dudas y ajustar el comportamiento de Git de forma consciente. Aplicar estas opciones correctamente facilita el trabajo cotidiano y favorece una experiencia de uso más organizada.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Capítulo 7: Personalización de Git. Disponible en: https://git-scm.com/book/es/v2/Personalización-de-Git